

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

*Московский институт электроники и математики им. А.Н.Тихонова
Департамент электронной инженерии*

«Архитектура Kubernetes»

Практическая работа №2
по курсу «Архитектура компьютера и Системное программирование»
студента образовательной программы бакалавриата
«Информационная Безопасность»

Выполнил: Студент группы

БИБ233 Гутников Д.А.

Преподаватель:

Балескин В.А.

1. Введение

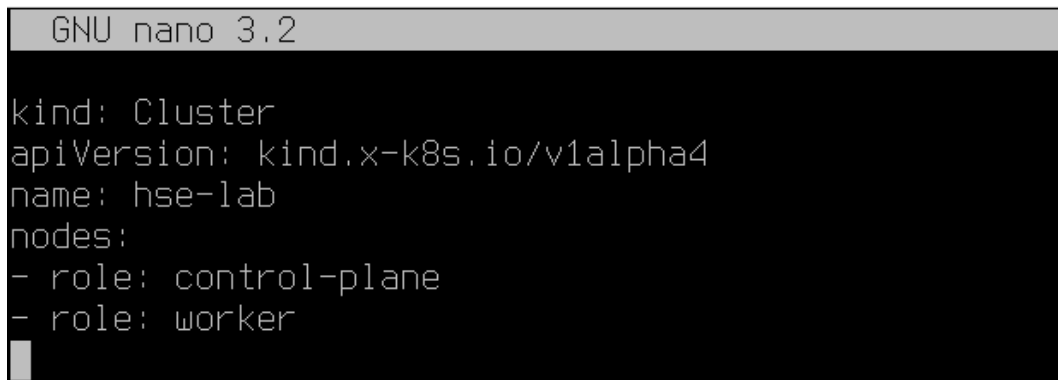
В данной лабораторной работе изучаются компоненты Kubernetes, создаются поды, приложения и другие объекты данной системы. Изучаются теоретические основы работы Kubernetes и его безопасность.

2. Основная часть

2.1 Путь запроса: от манифеста до контейнера

Обычно для работы кластера Kubernetes создают две ноды: control-plane и worker. На ноде control-plane располагаются статические поды, которые создаются из файлов-манифестов в /etc/kubernetes/manifests, а в worker-ноде располагаются kubelet и запускаемые им поды.

Напишем простой конфигурационный файл, который будет декларировать создание этих нод (рисунок 1).



```
GNU nano 3.2

kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: hse-lab
nodes:
- role: control-plane
- role: worker
```

Рисунок 1 – Конфигурационный файл кластера.

Чтобы создать кластер на основе конфигурационного файла, вызовем утилиту kind с указанием конфигурационного файла (рисунок 2).



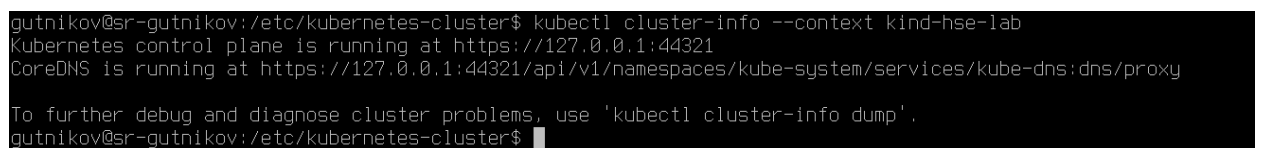
```
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$ kind create cluster --config hse-kind-config.yaml
Creating cluster "hse-lab" ...
 ✓ Ensuring node image (kindest/node:v1.35.0)
 ✓ Preparing nodes
 ✓ Writing configuration
 ✓ Starting control-plane
 ✓ Installing CNI
 ✓ Installing StorageClass
 ✓ Joining worker nodes
Set kubectl context to "kind-hse-lab"
You can now use your cluster with:

kubectl cluster-info --context kind-hse-lab

Not sure what to do next? ☺ Check out https://kind.sigs.k8s.io/docs/user/quick-start/
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$
```

Рисунок 2 – Создание кластера.

Убедимся, что созданный кластер работает (рисунок 3-4).



```
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$ kubectl cluster-info --context kind-hse-lab
Kubernetes control plane is running at https://127.0.0.1:44321
CoreDNS is running at https://127.0.0.1:44321/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$
```

Рисунок 3 – Получение информации о кластере.

```
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$ kubectl get nodes
NAME                                STATUS    ROLES    AGE      VERSION
hse-lab-control-plane              Ready    control-plane    8m58s    v1.35.0
hse-lab-worker                     Ready    <none>          8m44s    v1.35.0
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$
```

Рисунок 4 – Получение информации о запущенных нодах.

Создадим конфигурационный файл для пода hse-trace (рисунок 5) и применим его с помощью команды kubectl (рисунок 6).

```
GNU nano 3.2

apiVersion: v1
kind: Pod
metadata:
  name: hse-trace
  namespace: default
spec:
  containers:
  - name: hse-nginx
    image: nginx:alpine
  resources:
    requests:
      memory: "32Mi"
      cpu: "50m"
```

Рисунок 5 – Конфигурационный файл контейнера.

```
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$ kubectl apply -f hse-trace.yaml
pod/hse-trace created
gutnikov@sr-gutnikov:/etc/kubernetes-cluster$
```

Рисунок 6 – Создание пода hse-trace.

Теперь его можно увидеть при использовании утилиты crictl в ноде hse-lab-worker (рисунок 7). Здесь видно, что создан лишь один контейнер, однако на практике создаётся ещё один контейнер: pause-контейнер.

```
gutnikov@sr-gutnikov:~$ docker exec -it hse-lab-worker bash
root@hse-lab-worker:/# crictl ps
CONTAINER          IMAGE          CREATED          STATE      NAME          ATTEMPT     POD_ID          POD          NAMESPACE
350df144aaf74a    812d47f806db4  2 hours ago     Running    hse-nginx     0           964a8d791db73  hse-trace    default
5e5267d3d9895e    4921d7abdf1fa  2 hours ago     Running    kindnet-cni   0           bb18f22dc12fe  kindnet-vvmf4 kube-system
d5a049decaf3b     32652ff1bbe6b  2 hours ago     Running    kube-proxy    0           0d3c7e569aa33  kube-proxy-h7hun kube-system
root@hse-lab-worker:/# crictl pods
POD_ID            CREATED          STATE      NAME          NAMESPACE    ATTEMPT     RUNTIME
664a8d791db73    2 hours ago     Ready      hse-trace     default       0           (default)
bb18f22dc12fe    2 hours ago     Ready      kindnet-vvmf4 kube-system   0           (default)
0d3c7e569aa33    2 hours ago     Ready      kube-proxy-h7hun kube-system   0           (default)
root@hse-lab-worker:/#
```

Рисунок 7 – Обнаружение пода hse-trace.

С помощью параметра `inspect` и команды «`grep pid`» мы сможем получить информацию о PID нашего процесса в поде (в нашем случае это 1270), а если мы введём параметр `inspectr`, то мы сможем узнать PID pause-контейнера. Пространства имён NET и UTS у них должны совпадать, проверим это (рисунок 8).

```
root@hse-lab-worker:/# ls -la /proc/1244/ns/net
lrwxrwxrwx 1 65535 65535 0 Apr 29 11:38 /proc/1244/ns/net -> 'net:[40265324221]'
root@hse-lab-worker:/# ls -la /proc/1270/ns/net
lrwxrwxrwx 1 root root 0 Apr 29 13:52 /proc/1270/ns/net -> 'net:[40265324221]'
root@hse-lab-worker:/#
```

Рисунок 8 – Совпадение пространств имён NET.

Данный namespace был создан pause-контейнером. После создания он продолжает удерживать его и имеет возможность передавать его другим процессам, что и видно на практике. Операция передачи пространства имён идентична вызову функции `unshare` с параметром `--net`, равным значению `net_namespace`.

Посмотрим информацию о поде в `etcd` напрямую (рисунок 9). На первый взгляд, вывод имеет вид json, но если присмотреться, то видно, что это сериализованные данные, а именно `protobuf`.

```
root@hse-lab-control-plane:~# /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/63/fs/usr/local/bin/etcdctl \
> --endpoints=https://127.0.0.1:2379 \
> --cacert=/etc/kubernetes/pki/etcd/ca.crt \
> --cert=/etc/kubernetes/pki/etcd/server.crt \
> --key=/etc/kubernetes/pki/etcd/server.key \
> get /registry/pods/default/hse-trace | head -40
/registry/pods/default/hse-trace
0s
v1Pod
{
  "hse-trace": {
    "default": {
      "apiVersion": "v1",
      "kind": "Pod",
      "metadata": {
        "annotations": {},
        "name": "hse-trace",
        "namespace": "default"
      },
      "spec": {
        "containers": [
          {
            "image": "nginx:alpine",
            "name": "hse-nginx",
            "resources": {
              "requests": {
                "cpu": "50m",
                "memory": "32Mi"
              }
            }
          }
        ]
      },
      "status": {
        "conditions": [
          {
            "type": "ContainersReady",
            "status": "True",
            "lastProbeTime": null,
            "lastTransitionTime": null
          }
        ],
        "phase": "Running",
        "ip": "10.244.1.3",
        "hostIP": "10.244.1.3",
        "startTime": null
      }
    }
  }
}
```

Рисунок 9 – Сериализованные данные.

Несанкционированный доступ к порту 2379 (к etcd непосредственно) является большой проблемой, так как злоумышленник в таком случае получает доступ к секретам, конфигурациям подов, а также получает возможность создавать конфигурации новых подов, минуя API.

Общая цепочка от kubectl до создания контейнеров будет выглядеть следующим образом:

1. Отправка запроса на применение конфигурации нового пода hse-trace;
2. Приём запроса API-сервером, аутентификация, сохранение данных в etcd;
3. Deployment Controller создаёт ReplicaSet (в нашем случае это 1 под);
4. ReplicaSet Controller создаёт под в etcd;
5. Scheduler назначает имя ноды;
6. Kubelet обнаруживает новый под;
7. Kubelet через CRI создаёт pause-контейнер (containerd (dockerd) => runc => процесс (PID 1244));
8. Kubelet через CRI создаёт контейнер nginx:alpine (containerd (dockerd) => runc => процесс (PID 1270)).

2.2 Reconciliation loop: деструктивные эксперименты

Проверим механизм reconciliation loop, реализованный в Kubernetes. Для тестирования создадим конфигурацию объекта app с тремя репликами подов (рисунок 10). Применим конфигурацию (рисунок 11).

```
GNU nano 3.2
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hse-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: hse-app
  template:
    metadata:
      labels:
        app: hse-app
    spec:
      containers:
      - name: hse-nginx
        image: nginx:alpine
```

Рисунок 10 – Конфигурация hse-app.

```

gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl apply -f hse-app.yaml
deployment.apps/hse-app created
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,replicaset,pods -l app=hse-app
NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/hse-app-75cb6465bf  3          3          3        26s

NAME                                READY     STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-4zq9h        1/1       Running   0           26s
pod/hse-app-75cb6465bf-p2fn9        1/1       Running   0           26s
pod/hse-app-75cb6465bf-wggxz        1/1       Running   0           26s
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$

```

Рисунок 11 – Запуск hse-app.

Получилось так, что Deployment Controller создал ReplicaSet, в котором находилась информация о необходимых подах, а уже ReplicaSet Controller создал поды, которые требовались по конфигурации.

Попробуем произвести деструктивные действия по отношению к поду. Для наглядности запустим мониторинг подов (рисунок 12). Теперь можем удалить под (рисунок 13) и посмотреть в окно мониторинга (рисунок 14).

```

gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get pods -l app=hse-app -w
NAME                                READY     STATUS    RESTARTS   AGE
hse-app-75cb6465bf-4zq9h           1/1       Running   0           50s
hse-app-75cb6465bf-p2fn9           1/1       Running   0           50s
hse-app-75cb6465bf-wggxz           1/1       Running   0           50s

```

Рисунок 12 – Запуск мониторинга подов.

```

gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl delete pod/hse-app-75cb6465bf-4zq9h
pod "hse-app-75cb6465bf-4zq9h" deleted from default namespace
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,pods -l app=hse-app
NAME                                READY     STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-cwvhn        1/1       Running   0           41s
pod/hse-app-75cb6465bf-p2fn9        1/1       Running   0          6m54s
pod/hse-app-75cb6465bf-wggxz        1/1       Running   0          6m54s
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$

```

Рисунок 13 – Результат удаления пода.

```

gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get pods -l app=hse-app -w
NAME                                READY     STATUS    RESTARTS   AGE
hse-app-75cb6465bf-4zq9h           1/1       Running   0           50s
hse-app-75cb6465bf-p2fn9           1/1       Running   0           50s
hse-app-75cb6465bf-wggxz           1/1       Running   0           50s
hse-app-75cb6465bf-4zq9h           1/1       Terminating 0          6m13s
hse-app-75cb6465bf-cwvhn           0/1       Pending     0           0s
hse-app-75cb6465bf-4zq9h           1/1       Terminating 0          6m13s
hse-app-75cb6465bf-cwvhn           0/1       Pending     0           0s
hse-app-75cb6465bf-cwvhn           0/1       ContainerCreating 0           0s
hse-app-75cb6465bf-4zq9h           0/1       Completed   0          6m13s
hse-app-75cb6465bf-4zq9h           0/1       Completed   0          6m13s
hse-app-75cb6465bf-4zq9h           0/1       Completed   0          6m13s
hse-app-75cb6465bf-cwvhn           1/1       Running     0           1s

```

Рисунок 14 – Мониторинг состояния подов.

Как можно увидеть, вместо удалённого пода автоматически созданся новый под. Если изучить мониторинг подов, то здесь видно, что данный под прошёл через следующие стадии: ожидание, создание контейнера, запуск.

Теперь попробуем не удалить под, а снять с него метку, что он принадлежит приложению hse-app (рисунок 15). По выводу консоли можно сделать вывод, что при снятии лейбла под не уничтожается автоматически, а остаётся висеть. При этом на его место встанёт новый созданный под.

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ POD=$(kubectl get pod -l app=hse-app \
> -o jsonpath='{.items[0].metadata.name}')
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl label pod $POD app-
pod/hse-app-75cb6465bf-cwvhn unlabeled
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get pods -l app=hse-app
NAME                                READY   STATUS    RESTARTS   AGE
hse-app-75cb6465bf-5frjc            1/1     Running   0          35s
hse-app-75cb6465bf-p2fn9            1/1     Running   0          24m
hse-app-75cb6465bf-wggxz            1/1     Running   0          24m
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get pods --show-labels | grep $POD
hse-app-75cb6465bf-cwvhn            1/1     Running   0          18m      pod-template-hash=75cb6465bf
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 15 – Снятие метки с пода.

Selector в рамках ReplicaSet проверяет, что все поды функционируют, а если один из подов исчезает, то на его месте появляется новый под.

В control-plane размещены статические поды, которые включаются при наличии конфигурации-манифеста в /etc/kubernetes/manifests. Попробуем спрятать манифест scheduler'a (рисунок 16) и посмотреть на поведение kubernetes в нестандартной ситуации (рисунок 17).

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ docker exec hse-lab-control-plane \
> mv /etc/kubernetes/manifests/kube-scheduler.yaml \
> /tmp/kube-scheduler.yaml
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 16 – Соккрытие манифеста scheduler.

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,pods -l app=hse-app
NAME                                READY   STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-5frjc        1/1     Running   0          8m38s
pod/hse-app-75cb6465bf-p2fn9        1/1     Running   0          32m
pod/hse-app-75cb6465bf-wggxz        1/1     Running   0          32m
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl delete pod/hse-app-75cb6465bf-5frjc
pod "hse-app-75cb6465bf-5frjc" deleted from default namespace
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,pods -l app=hse-app
NAME                                READY   STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-59vhn        0/1     Pending   0          10s
pod/hse-app-75cb6465bf-p2fn9        1/1     Running   0          32m
pod/hse-app-75cb6465bf-wggxz        1/1     Running   0          32m
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 17 – Нестандартное поведение kubernetes.

ReplicaSet обнаружил, что не хватает одного пода и создал новый, но из-за того, что scheduler не работает, он не назначил новому поду значение nodeName. Без этого значения kubelet не может найти этот под и запустить на нём контейнер. Если же мы вернём scheduler обратно, то на поде запустится контейнер (рисунок 18).

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ docker exec hse-lab-control-plane \
> mv /tmp/kube-scheduler.yaml \
> /etc/kubernetes/manifests/kube-scheduler.yaml
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,pods -l app=hse-app
NAME                                READY   STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-59vhn        1/1     Running   0           7m8s
pod/hse-app-75cb6465bf-p2fn9        1/1     Running   0           39m
pod/hse-app-75cb6465bf-wggxz        1/1     Running   0           39m
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 18 – Возвращение scheduler.

Теперь аналогичным образом отключим control-manager, который в постоянном цикле проверяет соответствие текущего и желаемого состояний объектов (рисунок 19). Удалим под и посмотрим, как отреагирует система (рисунок 20).

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ docker exec hse-lab-control-plane \
> mv /etc/kubernetes/manifests/kube-controller-manager.yaml \
> /tmp/kube-controller-manager.yaml
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 19 – Отключение control-manager.

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl delete pod/hse-app-75cb6465bf-59vhn
pod "hse-app-75cb6465bf-59vhn" deleted from default namespace
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,pods -l app=hse-app
NAME                                READY   STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-p2fn9        1/1     Running   0           47m
pod/hse-app-75cb6465bf-wggxz        1/1     Running   0           47m
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 20 – Результат удаления пода.

Мы видим, что после удаления пода новый под не создался. Это произошло потому, что цикл проверки состояний остановился с отключением Controller Manager. Старые поды продолжают работу, потому что Controller Manager не влияет на уже запущенные поды. Если же мы вернём манифест Controller Manager назад (рисунок 21), то под создастся обратно (рисунок 22).

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ docker exec hse-lab-control-plane \
> mv /tmp/kube-controller-manager.yaml \
> /etc/kubernetes/manifests/kube-controller-manager.yaml
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 21 – Возврат манифеста Controller Manager.

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get deployment,pods -l app=hse-app
NAME                                READY   STATUS    RESTARTS   AGE
pod/hse-app-75cb6465bf-4547d        1/1     Running   0           42s
pod/hse-app-75cb6465bf-p2fn9        1/1     Running   0           49m
pod/hse-app-75cb6465bf-wggxz        1/1     Running   0           49m
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 22 – Создание нового пода.

По итогам наших действий заполним таблицу 1.

Таблица 1 – Результаты деструктивных экспериментов.

Компонент	Что ломается при его отказе	Что продолжает работать
scheduler	Новым подам не назначаются автоматически значение nodeName, контейнеры не запускаются	Уже запущенные контейнеры в рамках работающих подов
controller manager	Цикл проверки состояний перестает работать, автоматическое восстановление компонентов не срабатывает	Уже запущенные контейнеры в рамках работающих подов

2.3 Service и сетевая модель

Поды внутри кластера обнаруживают друг друга с помощью механизма Service Discovery. Создадим Service (рисунок 23).

```
GNU nano 3.2
apiVersion: v1
kind: Service
metadata:
  name: hse-svc
spec:
  selector:
    app: hse-app
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Рисунок 23 – Конфигурация Service.

Запустим service и посмотрим на его состояние (рисунок 24). По выводу команд мы получаем Cluster-IP=10.96.209.201 и адреса подов: 10.244.1.20, 10.244.1.22, 10.244.1.26.

```
gutnikov@sr-gutnikov:/etc/kubernetes-service$ kubectl apply -f hse-svc.yaml
service/hse-svc created
gutnikov@sr-gutnikov:/etc/kubernetes-service$ kubectl get svc hse-svc
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
hse-svc      ClusterIP     10.96.209.201 <none>         80/TCP     26s
gutnikov@sr-gutnikov:/etc/kubernetes-service$ kubectl get endpoints hse-svc
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME         ENDPOINTS                                           AGE
hse-svc      10.244.1.20:80,10.244.1.22:80,10.244.1.26:80     42s
gutnikov@sr-gutnikov:/etc/kubernetes-service$
```

Рисунок 24 – Запуск Service.

Создадим временный под и попробуем обратиться через него к кластеру (рисунок 25). Также мы можем обратиться через временный под к поду из hse-app напрямую по IP-адресу (рисунок 26).

```
gutnikov@sr-gutnikov:/etc/kubernetes-service$ kubectl run hse-client --image=alpine:3.19 \
> --rm -it --restart=Never -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
/ # wget -qO- http://hse-svc
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>
<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
/ #
```

Рисунок 25 – Обращение к кластеру через под.

Чтобы HTTP-запрос обработался кластером Kubernetes, нужно, чтобы DNS-запрос был обработан CoreDNS. Отсюда мы получим ClusterIP сервиса, откуда kube-проху перенаправит запрос на один из подов, если на них не будут стоять запрещающие правила iptables.

Обратимся теперь к поду напрямую без использования DNS (рисунок 26).


```

"kubernetes.io": {
  "namespace": "default",
  "node": {
    "name": "hse-lab-worker",
    "uid": "73048e04-71a3-41d8-9a18-ca13d8052924"
  },
  "pod": {
    "name": "hse-trace",
    "uid": "9b639eaa-a5b8-4388-94ac-e2ba2b482350"
  },
  "serviceaccount": {
    "name": "default",
    "uid": "0712c3b6-561b-43a5-93c7-bfbbc04386dc"
  },
  "warnafter": 1777492597
},
"nbf": 1777488990,
"sub": "system:serviceaccount:default:default"
}

```

Поле sub показывает, кому принадлежит токен (как уже было отмечено заранее, токен принадлежит default пользователю), а поле iss (issuer) показывает, кто выдал данный токен.

Выполним запрос к API-серверу с помощью данного токена (рисунок 28).

```

gutnikov@sr-gutnikov:~$ kubectl exec hse-trace -- sh -c \
> 'TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token);
> wget -qO- --no-check-certificate \
> --header="Authorization: Bearer $TOKEN" \
> https://kubernetes.default.svc/api/v1/namespaces/default/pods'
wget: server returned error: HTTP/1.1 403 Forbidden
command terminated with exit code 1
gutnikov@sr-gutnikov:~$ █

```

Рисунок 28 – Обращение к API с помощью токена.

В результате обращения по данному адресу мы получили код 403, который говорит о запрете обращения к данному адресу. Это последовательный ответ, потому что по пути /pods располагается чувствительная информация о подах, которая не должна быть доступна обычным пользователям. Default пользователю доступен ограниченный набор

прав только для чтения (рисунок 29). Если же рассмотреть доступ к /pods, то здесь для описания подойдёт рисунок 30: у администратора есть право просмотра данной конечной точки, а у пользователя default – нет.

```
gutnikov@sr-gutnikov:/etc/kubernetes-oom-pod$ kubectl auth can-i --list --as=serviceaccount:default:default
Resources      Non-Resource URLs  Resource Names      Verbs
selfsubjectreviews.authentication.k8s.io      []                  []                  [create]
selfsubjectaccessreviews.authorization.k8s.io []                  []                  [create]
selfsubjectrulesreviews.authorization.k8s.io   []                  []                  [create]
[]                                                []                  [get]
[/api/*]                                         []                  [get]
[/api]                                           []                  [get]
[/apis/*]                                        []                  [get]
[/apis]                                          []                  [get]
[/healthz]                                      []                  [get]
[/healthz]                                      []                  [get]
[/livez]                                        []                  [get]
[/livez]                                        []                  [get]
[/openapi/*]                                    []                  [get]
[/openapi]                                      []                  [get]
[/readyz]                                       []                  [get]
[/readyz]                                       []                  [get]
[/version/]                                    []                  [get]
[/version/]                                    []                  [get]
[/version]                                      []                  [get]
[/version]                                      []                  [get]
```

Рисунок 29 – Права default.

```
gutnikov@sr-gutnikov:~$ kubectl auth can-i get pods
yes
gutnikov@sr-gutnikov:~$ kubectl auth can-i get pods --as=system.serviceaccount:default:default
no
gutnikov@sr-gutnikov:~$ █
```

Рисунок 30 – Демонстрация прав на доступ к /pods.

Создадим секрет в etcd (рисунок 31). Данный секрет нам понадобится для демонстрации получения доступа из не аутентифицированных мест.

```
gutnikov@sr-gutnikov:~$ kubectl create secret generic hse-secret \
> --from-literal=password=hse-super-secret-2026
secret/hse-secret created
gutnikov@sr-gutnikov:~$ █
```

Рисунок 31 – Создание секрета.

Покажем, что можно получить доступ к паролю в etcd в control-plane ноде (рисунок 32). Данный пароль оказался не зашифрован (и даже не закодирован в base64), что представляет угрозу конфиденциальности данных. Защитить секреты в Kubernetes позволяет Encryption at Rest.

Таблица 2 – Что было обнаружено.

Вектор	Что обнаружено	Угроза
SA токен	Автоматически монтируемый токен для пользователя serviceaccount:default:default представлен в открытом виде	Возникновение проблемы аутентификации при неправильной конфигурации. Возможна кража JWT-токена и извлечение из него чувствительной информации
etcd	Данные в etcd по умолчанию хранятся в незашифрованном виде	Утечка конфиденциальной информации, представленной в незашифрованном виде
kubelet:10250	Возможность обратиться к kubelet напрямую из ноды control-plane	Произвольное исполнение кода при получении доступа к kubelet без проверок API

2.5 Эксперименты с компонентами кластера

Проведём эксперименты с компонентами кластера и сделаем сравнительный анализ объектов Kubernetes и примитивов Linux, которые используются для создания контейнеров.

Изучим сетевое пространство имён. Посмотрим на под hse-trace и изучим два контейнера, которые находятся в нём: pause-контейнер и nginx:alpine. Проверим, какие сетевые интерфейсы указаны у данных контейнеров (рисунок 35 и 36). Видно, что сетевые интерфейсы совпадают полностью. Это связано с тем, что pause-контейнер создаёт пространство имён, а затем удерживает его, чтобы им можно было поделиться с целевым контейнером.

```
root@hse-lab-worker: /# nsenter -t 1244 ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if5          UP           172.18.0.2/16 fc00:f853:ccd:e793::2/64 fe80::f094:59ff:fe03:2dca/64
vethaba09b6d@if2 UP           10.244.1.1/32 fe80::68fe:53ff:fe25:9865/64
veth7c6e28cd@if2 UP           10.244.1.1/32 fe80::c9e:8aff:fe34:f9c9/64
vethc088ab38@if2 UP           10.244.1.1/32 fe80::4827:a9ff:fe8f:512f/64
vethe67282e2@if2 UP           10.244.1.1/32 fe80::c071:8fff:fe53:7a8d/64
veth551d2715@if2 UP           10.244.1.1/32 fe80::4867:bfff:feb8:6137/64
root@hse-lab-worker: /#
```

Рисунок 35 – Сетевые интерфейсы pause-контейнера.

```
root@hse-lab-worker: /# nsenter -t 1270 ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0@if5          UP           172.18.0.2/16 fc00:f853:ccd:e793::2/64 fe80::f094:59ff:fe03:2dca/64
vethaba09b6d@if2 UP           10.244.1.1/32 fe80::68fe:53ff:fe25:9865/64
veth7c6e28cd@if2 UP           10.244.1.1/32 fe80::c9e:8aff:fe34:f9c9/64
vethc088ab38@if2 UP           10.244.1.1/32 fe80::4827:a9ff:fe8f:512f/64
vethe67282e2@if2 UP           10.244.1.1/32 fe80::c071:8fff:fe53:7a8d/64
veth551d2715@if2 UP           10.244.1.1/32 fe80::4867:bfff:feb8:6137/64
root@hse-lab-worker: /#
```

Рисунок 36 – Сетевые интерфейсы hse-trace.

Покажем, что после завершения процесса целевого контейнера (PID 1270) pause-контейнер передаст новому контейнеру ту же самую информацию о сетевых ресурсах (рисунок 37-39). Как можно увидеть, адреса остались теми же, потому что контейнер получил тот же самый namespace, который был у предшествующего контейнера.

```
root@hse-lab-worker:~# kill -9 1270
root@hse-lab-worker:~# crictl ps
```

CONTAINER SPACE	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID	POD	NAME
8e139449fef08	812d47f806db4	47 seconds ago	Running	hse-nginx	1	964a8d791db73	hse-trace	defa
9da1c3aab8338	812d47f806db4	4 hours ago	Running	hse-nginx	0	297ab43dfc7ec	hse-app-75cb6465bf-4547d	defa
016d5104bd58e	812d47f806db4	4 hours ago	Running	hse-nginx	0	e3d2e314e25f1	hse-app-75cb6465bf-cuvhn	defa
aaf118e283f59	812d47f806db4	5 hours ago	Running	hse-nginx	0	e2b4705c06d28	hse-app-75cb6465bf-p2fn9	defa
dd5b162abad3f	812d47f806db4	5 hours ago	Running	hse-nginx	0	88c771a5b3ac7	hse-app-75cb6465bf-wggxz	defa
5e526fd4a896e	4921d7a6dffa9	10 hours ago	Running	kindnet-cn1	0	bb18f22dc12fe	kindnet-vvmf4	kube
d5a049decaf3b	32652ff1bbe6b	10 hours ago	Running	kube-proxy	0	0d3c7e569aa33	kube-proxy-h7hun	kube

```
root@hse-lab-worker:~#
```

Рисунок 37 – Завершение процесса и появление нового контейнера.

```
root@hse-lab-worker:~# crictl inspect 8e139449fef08 | grep pid
  "pid": 1,
  "pid": 14004,
  "type": "pid"
root@hse-lab-worker:~# crictl inspect 964a8d791db73 | grep pid
  "pid": 1,
  "pid": 1244,
  "type": "pid"
    "getpid",
    "getppid",
    "pidfd_open",
    "pidfd_send_signal",
    "waitpid",
    "pid": 1,
    "pid": "CONTAINER",
root@hse-lab-worker:~#
```

Рисунок 38 – Соответствие контейнеров в поде.

```
root@hse-lab-worker:~# nsenter -t 14004 ip -br a
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0@if5 UP 172.18.0.2/16 fc00:f853:ccd:e793::2/64 fe80::fc094:59ff:fe03:2dca/64
vethaba09b6d@if2 UP 10.244.1.1/32 fe80::68fe:53ff:fe25:9865/64
veth7c6e28cd@if2 UP 10.244.1.1/32 fe80::c9e:8aff:fe34:f9c9/64
vethc088ab38@if2 UP 10.244.1.1/32 fe80::4827:a9ff:fe8f:512f/64
veth657282e2@if2 UP 10.244.1.1/32 fe80::c071:8fff:fe53:7a8d/64
veth551d2715@if2 UP 10.244.1.1/32 fe80::4867:bfff:feb8:6137/64
root@hse-lab-worker:~#
```

Рисунок 39 – Оставшиеся прежними адреса в поде.

Теперь создадим статический под, написав манифест в /etc/kubernetes/manifests (рисунок 40).

```
gutnikov@sr-gutnikov:~$ docker exec -it hse-lab-control-plane bash
root@hse-lab-control-plane:~# cat > /etc/kubernetes/manifests/hse-static.yaml <<EOF
> apiVersion: v1
> kind: Pod
> metadata:
>   name: hse-static
>   namespace: default
> spec:
>   containers:
>   - name: hse-busybox
>     image: busybox
>     command: ["sleep", "3600"]
> EOF
root@hse-lab-control-plane:~#
```

Рисунок 40 – Создание статического пода в control-plane.

Если посмотреть на поды в control-plane, то там появится новый под, который мы назначили только что (рисунок 41).

```
root@hse-lab-control-plane:/# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
hse-app-75cb6465bf-4547d	1/1	Running	0	3h55m
hse-app-75cb6465bf-cuvhn	1/1	Running	0	4h38m
hse-app-75cb6465bf-p2fn9	1/1	Running	0	4h44m
hse-app-75cb6465bf-wgqxz	1/1	Running	0	4h44m
hse-static-hse-lab-control-plane	1/1	Running	0	3m15s
hse-trace	1/1	Running	1 (14m ago)	9h

```
root@hse-lab-control-plane:/# crictl ps
```

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID
hse-lab-control-plane	hse-lab-control-plane	3 minutes ago	Running	hse-busybox	0	3cfb4d1e
hse-lab-control-plane	hse-lab-control-plane	4 hours ago	Running	kube-controller-manager	0	8b85eab4
hse-lab-control-plane	hse-lab-control-plane	4 hours ago	Running	kube-scheduler	0	2d99e65b
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	local-path-provisioner	0	b34c35ef
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	coredns	0	1a0eae5b
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	coredns	0	af30fc8c
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	kindnet-cni	0	8755fbf4
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	kube-proxy	0	a0d8ae09
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	etcd	0	8756dc67
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	kube-apiserver	0	87835341

Рисунок 41 – Появление статичекого пода.

Если мы удалим под (рисунок 42), то он создастся заново (рисунок 43).

```
root@hse-lab-control-plane:/# kubectl delete pod hse-static-hse-lab-control-plane
pod "hse-static-hse-lab-control-plane" deleted from default namespace
root@hse-lab-control-plane:/#
```

Рисунок 42 – Удаление пода.

```
root@hse-lab-control-plane:/# crictl ps
```

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID	POD
hse-lab-control-plane	hse-lab-control-plane	12 minutes ago	Running	hse-busybox	0	3cfb4d1e01e95	hse-static-hse-lab-control-plane
hse-lab-control-plane	hse-lab-control-plane	4 hours ago	Running	kube-controller-manager	0	8b85eab47f8d5	kube-controller-manager-hse-lab-control-plane
hse-lab-control-plane	hse-lab-control-plane	4 hours ago	Running	kube-scheduler	0	2d99e65bab8c0	kube-scheduler-hse-lab-control-plane
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	local-path-provisioner	0	b34c35ef25eec	local-path-provisioner-67hse-lab-control-plane
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	coredns	0	1a0eae5b0756d	coredns-7d764666f9-7utvt
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	coredns	0	af30fc8c0c1dd	coredns-7d764666f9-hvdkw
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	kindnet-cni	0	8755fbf4299d7	kindnet-fflxx
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	kube-proxy	0	a0d8ae09ccd95	kube-proxy-z978n
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	etcd	0	8756dc679111c	etcd-hse-lab-control-plane
hse-lab-control-plane	hse-lab-control-plane	10 hours ago	Running	kube-apiserver	0	87835341eb953	kube-apiserver-hse-lab-control-plane

```
root@hse-lab-control-plane:/#
```

Рисунок 43 – Под создался заново.

Если же мы удалим манифест данного пода (рисунок 44), то в этом случае он исчезнет (рисунок 45).

```
root@hse-lab-control-plane:/# rm /etc/kubernetes/manifests/hse-static.yaml
root@hse-lab-control-plane:/#
```

Рисунок 44 – Удаление манифеста.

```
root@hse-lab-control-plane:/# crictl ps
```

CONTAINER	IMAGE	CREATED	STATE	NAME	ATTEMPT	POD ID	POD
34e280cd7cc3b	2c9a4b058bd7e	4 hours ago	Running	kube-controller-manager	0	8b85eab47f8d5	kube-contr
iller-manager-hse-lab-control-plane	kube-system						
7f8b974033e25	550794e3b12ac	4 hours ago	Running	kube-scheduler	0	2d99e65bab8c0	kube-sched
iller-hse-lab-control-plane	kube-system						
ad8367c65c13b	65232e320e58b	10 hours ago	Running	local-path-provisioner	0	b34c35ef25eec	local-path
-provisioner-67b8995b4b-759x6	local-path-storage						
97d4659c718d8	aa5e3ebc0dfed	10 hours ago	Running	coredns	0	1a0eae5b0756d	coredns-7d
764666f9-7wtvt	kube-system						
fe2f42acbf27e	aa5e3ebc0dfed	10 hours ago	Running	coredns	0	af30fc8c0c1dd	coredns-7d
764666f9-hvdkw	kube-system						
9f079ea04251f	4921d7a6dffa9	10 hours ago	Running	kindnet-cni	0	8755fbf4299d7	kindnet-ff
lxx	kube-system						
30ed5f9e00e22	32652ff1bbe6b	10 hours ago	Running	kube-proxy	0	a0d8ae09ccd95	kube-proxy
-z978n	kube-system						
7814776b20c09	0a108f7189562	10 hours ago	Running	etcd	0	8756dc679111c	etcd-hse-l
ab-control-plane	kube-system						
2d61535ce6d87	5c6acd67e9cd1	10 hours ago	Running	kube-apiserver	0	878335341eb953	kube-apise
erver-hse-lab-control-plane	kube-system						

```
root@hse-lab-control-plane:/#
```

Рисунок 45 – Под исчез.

В случае статических подов, в отличие от рассмотренных ранее подов, владельцем является нода control-plane. Под не удаляется командой терминала, потому что из-за постоянной проверки текущего и желаемого результатов kubelet постоянно воссоздаёт под, чтобы состояние соответствовало манифестам. Прямой доступ злоумышленника к манифестам угрожает безопасности Kubernetes, потому что в таком случае он может запустить любой под с произвольным кодом.

Теперь попробуем применить механизмы cgroups по отношению к контейнеру. Создадим новый под с ограничением ресурсов (рисунок 46). Запустим новый под (рисунок 47).

```
GNU nano 3.2
apiVersion: v1
kind: Pod
metadata:
  name: hse-oom
spec:
  containers:
  - name: hse-stress
    image: alpine:3.19
    command: ["sleep", "3600"]
    resources:
      limits:
        memory: "32Mi"
```

Рисунок 46 – Создание конфигурации пода.

```
gutnikov@sr-gutnikov:/etc/kubernetes-oom-pod$ kubectl apply -f hse-oom-pod.yaml
pod/hse-oom created
gutnikov@sr-gutnikov:/etc/kubernetes-oom-pod$
```

Рисунок 47 – Запуск пода.

Теперь попробуем превысить пределы доступа к ресурсам (рисунок 48). Для превышения воспользуемся функцией, которая будет брать символы из /dev/zero и передавать их в /dev/null, но при этом передадим символы одним батчем по 64 мб.

```

gutnikov@sr-gutnikov:/etc/kubernetes-oom-pod$ kubectl exec hse-oom -- sh -c "dd if=/dev/zero of=/dev/null bs=64M count=1"
command terminated with exit code 137
gutnikov@sr-gutnikov:/etc/kubernetes-oom-pod$ kubectl describe pod hse-oom
Name: hse-oom
Namespace: default
Priority: 0
Service Account: default
Node: hse-lab-worker/172.18.0.2
Start Time: Thu, 30 Apr 2025 00:34:13 +0300
Labels: <none>
Annotations: <none>
Status: Running
IP: 10.244.1.28
IPs:
  IP: 10.244.1.28
Containers:
  hse-stress:
    Container ID: containerd://1d393931b9cd11bcd7b6a26963b9a835aa48ad4da624394d0a663b2ebd20c0ac
    Image: alpine:3.19
    Image ID: docker.io/library/alpine@sha256:6baf43584bcb78f2e5847d1de515f23499913ac9f12bdf834811a3145eb11ca1
    Port: <none>
    Host Port: <none>
    Command:
      sleep
      3600
    State: Running
      Started: Thu, 30 Apr 2025 00:46:35 +0300
    Last State: Terminated
      Reason: OOMKilled
      Exit Code: 137
      Started: Thu, 30 Apr 2025 00:34:13 +0300
      Finished: Thu, 30 Apr 2025 00:46:34 +0300
    Ready: True
    Restart Count: 1
    Limits:
      memory: 32Mi
    Requests:

```

Рисунок 48 – Превышение ограничений ресурсов.

Если мы изучим статистику данного пода, то мы сможем обнаружить то, что данный под перезапустился один раз.

Если мы зайдём в процесс данного контейнера (предварительно узнав его PID), то мы сможем найти те самые ограничения, которые мы указывали в конфигурации (рисунок 49). Число представляет из себя ограничение на использование оперативной памяти не более чем на 33554432 байта, что при переводе на мегабайты равняется 32.

```

root@hse-lab-worker:/# cat /proc/14870/cgroup
0::/kubelet.slice/kubelet-kubepods.slice/kubelet-kubepods-burstable.slice/kubelet-kubepods-burstable-podd39964f5_16ae_4464_975e_433d72b0a23b.slice/cr1-conta
inerd-1d393931b9cd11bcd7b6a26963b9a835aa48ad4da624394d0a663b2ebd20c0ac.scope
root@hse-lab-worker:/# cat /sys/fs/cgroup/kubelet.slice/kubelet-kubepods.slice/kubelet-kubepods-burstable.slice/kubelet-kubepods-burstable-podd39964f5_16ae
_4464_975e_433d72b0a23b.slice/cr1-containerd-1d393931b9cd11bcd7b6a26963b9a835aa48ad4da624394d0a663b2ebd20c0ac.scope/memory.max
33554432
root@hse-lab-worker:/#

```

Рисунок 49 – Значение ограничения памяти.

Ограничения cgroup призваны ограничивать потребление ресурсов подами. Это сделано для того, чтобы один процесс не забирал все ресурсы машины, когда эти ресурсы разделяются между несколькими объектами (решается проблема noisy neighbor).


```

/ # curl -k https://kubernetes.default.svc/version
{
  "major": "1",
  "minor": "35",
  "emulationMajor": "1",
  "emulationMinor": "35",
  "minCompatibilityMajor": "1",
  "minCompatibilityMinor": "34",
  "gitVersion": "v1.35.0",
  "gitCommit": "66452049f3d692768c39c797b21b793dce80314e",
  "gitTreeState": "clean",
  "buildDate": "2025-12-17T12:32:07Z",
  "goVersion": "go1.25.5",
  "compiler": "gc",
  "platform": "linux/amd64"
}
/ # curl -k -H "Authorization: Bearer $TOKEN" https://kubernetes.default.svc/version
{
  "major": "1",
  "minor": "35",
  "emulationMajor": "1",
  "emulationMinor": "35",
  "minCompatibilityMajor": "1",
  "minCompatibilityMinor": "34",
  "gitVersion": "v1.35.0",
  "gitCommit": "66452049f3d692768c39c797b21b793dce80314e",
  "gitTreeState": "clean",
  "buildDate": "2025-12-17T12:32:07Z",
  "goVersion": "go1.25.5",
  "compiler": "gc",
  "platform": "linux/amd64"
}
/ # █

```

Рисунок 53 –Проверка доступа к /version.

```

/ # curl -k https://kubernetes.default.svc/metrics
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "forbidden: User \"system:anonymous\" cannot get path \"/metrics\"",
  "reason": "Forbidden",
  "details": {},
  "code": 403
}
/ # curl -k -H "Authorization: Bearer $TOKEN" https://kubernetes.default.svc/metrics
{
  "kind": "Status",
  "apiVersion": "v1",
  "metadata": {},
  "status": "Failure",
  "message": "forbidden: User \"system:serviceaccount:default:default\" cannot get path \"/metrics\"",
  "reason": "Forbidden",
  "details": {},
  "code": 403
}
/ # █

```

Рисунок 54 – Проверка доступа к /metrics.

В рамках данных изображений можно сделать законченный вывод о доступе к данным конечным точкам в таблице 3.

Таблица 3 – Характеристики конечных точек

Конечная точка	Доступ к ней	Что содержит
/api	Ограниченный, только по токену	Информацию о входящих в группу ресурсах
/healthz	Неограниченный	Состояние кластера
/version	Неограниченный	Версия Kubernetes
/metrics	Отсутствует	Метрики компонентов control-plane

3.2 Сетевая достижимость подов

Попробуем обратиться к компонентам нод из пода с помощью команд `curl` и `nc`. Чтобы это сделать, узнаем адреса нод, чтобы потом обратиться к их компонентам по портам (рисунок 55). Проверим, что адреса действительно пингуются (рисунок 56). Покажем, как проходит маршрут от пода к ноде (рисунок 57).

```
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$ kubectl get nodes -o wide
NAME                STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE              KERNEL-VERSION   CONTAINER-RUNTIME
hse-lab-control-plane Ready    control-plane 27h   v1.35.0   172.18.0.3    <none>        Debian GNU/Linux 12 (bookworm) 5.15.0-70-generic containerd://2.2.0
hse-lab-worker       Ready    <none>      27h   v1.35.0   172.18.0.2    <none>        Debian GNU/Linux 12 (bookworm) 5.15.0-70-generic containerd://2.2.0
gutnikov@sr-gutnikov:/etc/kubernetes-deployment$
```

Рисунок 55 – Получение адресов нод.

```
/ # ping 172.18.0.3
PING 172.18.0.3 (172.18.0.3): 56 data bytes
64 bytes from 172.18.0.3: seq=0 ttl=63 time=0.285 ms
64 bytes from 172.18.0.3: seq=1 ttl=63 time=0.149 ms
64 bytes from 172.18.0.3: seq=2 ttl=63 time=0.294 ms
64 bytes from 172.18.0.3: seq=3 ttl=63 time=0.233 ms
^C
--- 172.18.0.3 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.149/0.240/0.294 ms
/ #
```

Рисунок 56 – Пинг к нодам.

```
/ # traceroute 172.18.0.3
traceroute to 172.18.0.3 (172.18.0.3), 30 hops max, 46 byte packets
 1  10.244.1.1 (10.244.1.1)  0.007 ms  0.004 ms  0.002 ms
 2  172-18-0-3.kubernetes.default.svc.cluster.local (172.18.0.3)  0.002 ms  0.003 ms  0.001 ms
/ #
```

Рисунок 57 – Traceroute к ноде.

Попробуем обратиться к `etcd` по порту 2379 (рисунок 58). Здесь видно, что в результате выполнения первой команды мы узнали о том, что `etcd` доступен через под (его порт открыт), однако при обращении к нему по протоколу `HTTPS` сработала защита, которая потребовала наличие сертификата со стороны клиента (стоит `mTLS`).

```
/ # nc -zv 172.18.0.3 2379
172.18.0.3 (172.18.0.3:2379) open
/ # curl -k https://172.18.0.3:2379
curl: (56) SSL certificate OpenSSL verify result: unable to get local issuer certificate (20)
/ #
```

Рисунок 58 – Проверка доступности `etcd`.

Теперь попробуем обратиться к kubelet (рисунок 59). Проверка показала, что порт открыт. Если мы попробуем обратиться к данному компоненту через curl, то вернётся код 404, который говорит об отсутствии такой страницы: есть другие страницы, к которым мы можем попробовать получить доступ (рисунок 60): здесь уже другой код ошибки – неавторизованный.

```
/ # nc -zv 172.18.0.2 10250
172.18.0.2 (172.18.0.2:10250) open
/ # curl -k https://172.18.0.2:10250
404 page not found
/ #
```

Рисунок 59 – Проверка доступности kubelet.

```
/ # curl -k https://172.18.0.2:10250/pods
Unauthorized/ #
/ # curl -k https://172.18.0.2:10250/metrics
Unauthorized/ #
```

Рисунок 60 – Проверка доступности kubelet 2.

Попробуем авторизоваться с помощью токена default-пользователя и посмотрим на результат работы (рисунок 61). Теперь высвечивается третий код, который говорит о том, что нам запрещён доступ к данному компоненту (ограничения со стороны RBAC), несмотря на возможность обратиться к другим конечным точкам (рисунок 29).

```
/ # TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
/ # curl -k -H "Authorization: Bearer $TOKEN" https://172.18.0.2:10250/pods
Forbidden (user=system:serviceaccount:default:default, verb=get, resource=nodes, subresource(s)=[pods proxy])
/ # curl -k -H "Authorization: Bearer $TOKEN" https://172.18.0.2:10250/healthz
Forbidden (user=system:serviceaccount:default:default, verb=get, resource=nodes, subresource(s)=[healthz proxy])
/ #
```

Рисунок 61 – Запрет доступа к конечным точкам.

3.3 Attack narrative and kill chain

В данном опыте была проведена разведка периметра при наличии доступа к поду и с токеном аутентификации пользователя default. Целью данной разведки было обнаружение проблем конфигурации, которые могли бы привести к получению чувствительной информации или к возможности создания произвольных подов.

Для начала достанем токен аутентификации пользователя, который нам доступен – пользователя default. Проверим доступ к конечным точкам, потому что они часто проще всего достижимы, но при этом информативны: они могут дать очень много полезной информации как о поде, так и о кластере в целом. В таблице 4 покажем, какие конечные точки смогли дать нам полезную информацию.

Таблица 4 – Проверка конечных точек.

Конечная точка	Доступ	Риски
/api	Только при наличии аутентификации пользователя. Пользователь anonymous не получит доступ к данной конечной точке (код 200)	Злоумышленник не может навредить кластеру Kubernetes из этой конечной точки, но может получить информацию о компонентах системы для разведки
/healthz	Доступ есть как у пользователей, прошедших аутентификацию, так и у тех, кто не прошёл её (код 200)	Минимальные. Данная контрольная точка лишь показывает, работает API или нет. Следующей целью будет попытка добраться до etcd, если API работает
/version	Доступ есть как у пользователей, прошедших аутентификацию, так и у тех, кто не прошёл её (код 200)	Почти минимальные. Версия Kubernetes может раскрыть, какие уязвимости есть в текущем реализованном кластере (CVE)
/metrics	Отсутствует, даже несмотря на аутентификацию пользователя serviceaccount:default (код 403)	При получении доступа к данному эндпоинту злоумышленник получает информацию о состоянии подов и о потребляемых ресурсах этих подов. На основе полученной статистики он может сделать выводы о системе или реализовать специфичные атаки на компоненты

Другой особенностью Kubernetes, откуда могут исходить опасности, является сетевое взаимодействие подов, нодов и других компонентов кластера. Получив информацию об адресах нод, мы можем теперь обратиться к компонентам этих нод через специальные порты. Проверим связность и опишем наше взаимодействие с данными компонентами в таблицах 5 и 6.

Таблица 5 – Взаимодействие с etcd.

Вид взаимодействия	Доступ	Угроза
Через netcat	Прямой, порт открыт, есть возможность отправлять TCP-пакеты	Минимальная
Через curl	Отсутствует, так как не был найден подходящий сертификат для данного компонента, а без него не получается пройти через mTLS	Минимальная-средняя: если злоумышленник получит доступ к сертификату, то он сможет обращаться к etcd и вносить в него изменения в обход API

Таблица 6 – Взаимодействие с kubelet пода.

Вид взаимодействия	Доступ	Угроза
Через netcat	Прямой, порт открыт, есть возможность отправлять TCP-пакеты	Минимальная
Через curl без токена	Отсутствует, пользователь не авторизируется в компоненте	Минимальная: без токена при обычной конфигурации не будет возможности обратиться к конечным точкам
Через curl с токеном	Отсутствует, JWT-токен serviceaccount:default не даёт прав для взаимодействия с конечными точкам kubelet	Минимальная-средняя: заполучив JWT-токен, злоумышленник сможет получить доступ к чувствительной информации в kubelet

Уточним, из-за чего именно не получилось продвинуться дальше во время разведки (таблица 7).

Таблица 7 – Результат взаимодействия с компонентами.

Компонент	Что было обнаружено	Архитектурный барьер
Эндпоинты	Были как открытые конечные точки, так и закрытые. Закрытые точки выдавали код 403	RBAC, необходимость привилегированных ролей
etcd	При попытке обратиться к etcd он отправлял ошибку, о необходимости клиентского сертификата	mTLS, необходимость сертификата с клиента
Kubelet на поде	При отсутствии аторизации kubelet возвращал код 401, а при попытке войти с токеном возвращался код 403	RBAC, необходимость привилегированных ролей

3.4 Изменение конфигурации кластера

Для уменьшения поверхности атаки уберём автоматически монтируемый токен из пода (рисунок 62). Проверим, что в созданном поде исчез автоматически монтируемый токен (рисунок 63).

```
GNU nano 3.2

apiVersion: v1
kind: Pod
metadata:
  name: hse-trace-safer
  namespace: default
spec:
  automountServiceAccountToken: false
  containers:
  - name: hse-nginx
    image: nginx:alpine
    resources:
      requests:
        memory: "32Mi"
        cpu: "50m"
```

Рисунок 62 – Улучшенная реализация пода.

```
gutnikov@sr-gutnikov:/etc$ kubectl apply -f hse-trace-safer.yaml
pod/hse-trace-safer created
gutnikov@sr-gutnikov:/etc$ kubectl exec -it hse-trace-safer -- sh
/ # cat /var/run/secrets/kubernetes.io/serviceaccount/token
cat: can't open '/var/run/secrets/kubernetes.io/serviceaccount/token': No such file or directory
/ # █
```

Рисунок 63 – Отсутствие автомонтируемого токена.

Теперь, когда у нас пропадает автоматически монтируемый токен, уменьшается число возможных векторов атак и, впоследствии, повышается безопасность системы. Данное изменение не только закрывает некоторые проблемы, но и не затрагивает критические компоненты Kubernetes.

4. Вывод

По итогам работы освоены следующие практические навыки:

- Исследования внутреннего устройства кластера Kubernetes на уровне процессов, пространств имён и хранилища состояний;
- Практического понимания декларативной модели управления через деструктивные эксперименты с reconciliation loop;
- Анализа сетевой модели кластера и механизмов Service Discovery;
- Обнаружения поверхности атаки кластера Kubernetes в конфигурации по умолчанию;
- Связи механизмов Kubernetes с примитивами ядра Linux, изученными в ПР 4.1 (namespaces, cgroups, OverlayFS).

Полученные знания являются основой для дальнейшего изучения механизмов защиты кластера: сетевых политик (лекция 3), RBAC и Admission Control (лекция 4), безопасности узлов (лекция 5) и защиты цепочки поставки (лекция 6).